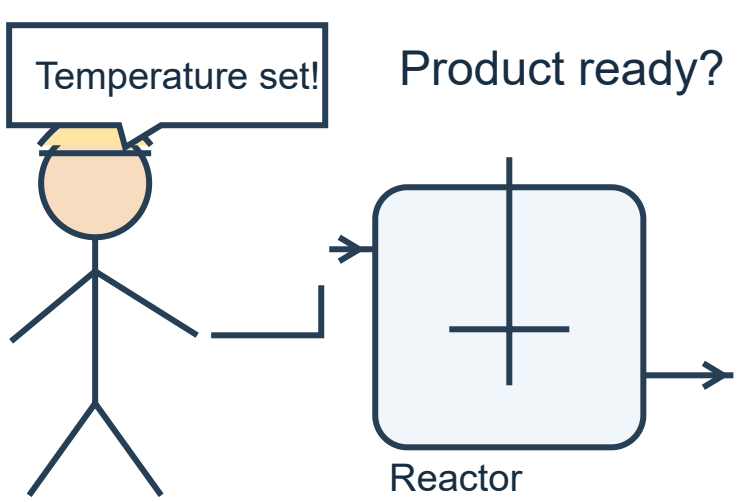
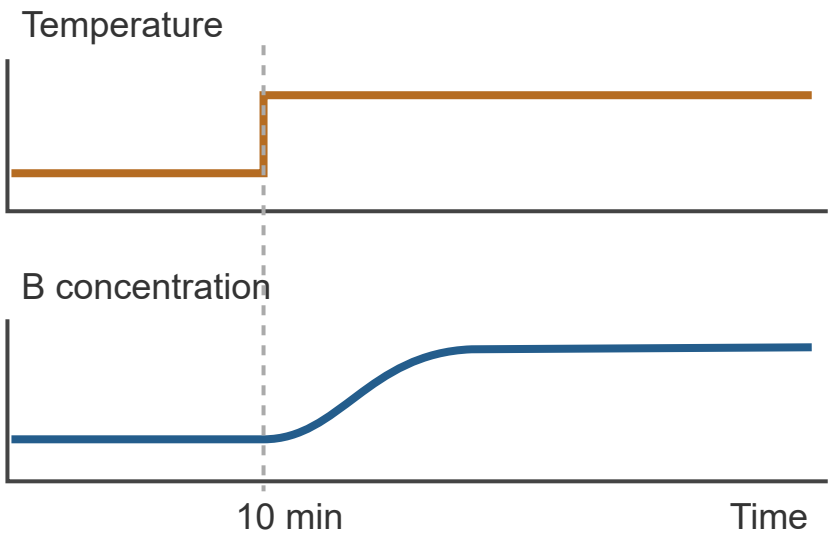
Week 3: Predicting Dynamic Responses and Rollout

Transient product concentrations after a temperature change

A process engineer changes reactor temperature at 10 min. If the outlet flows directly into a product tank, how will B concentration change over the next 20 min?

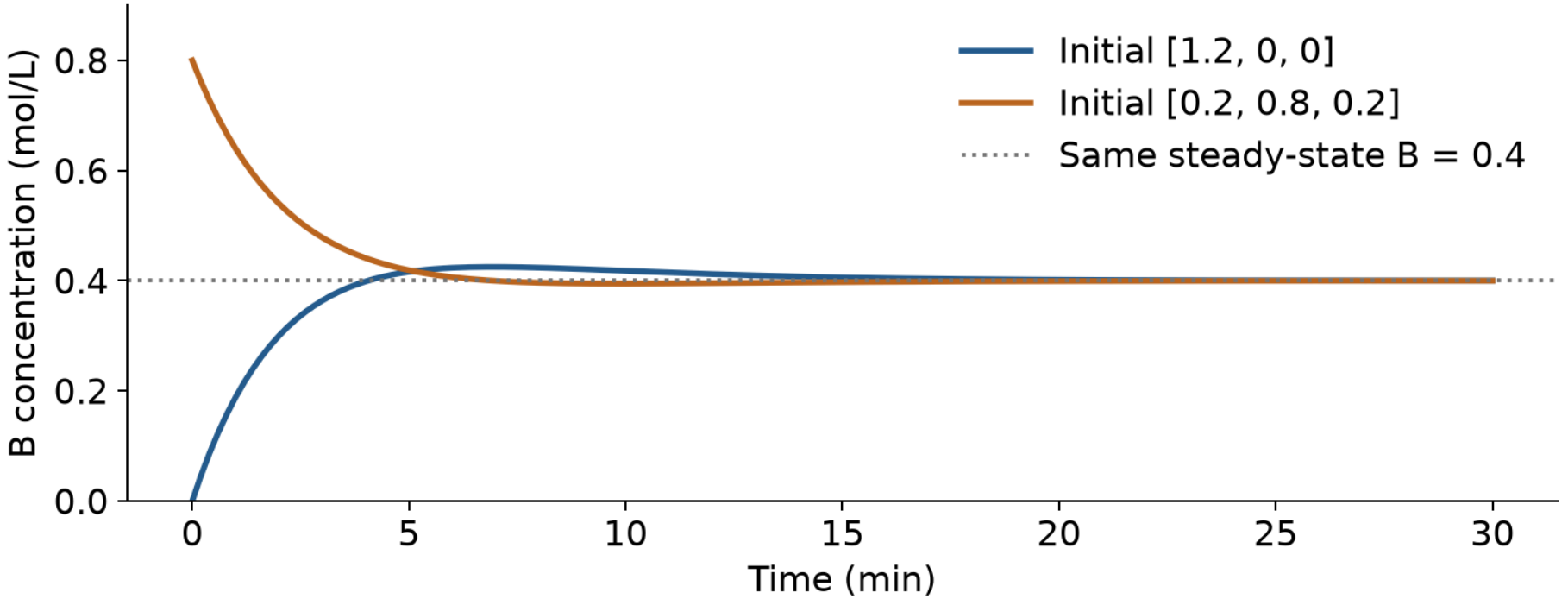




Transient response toward a new steady state

During a grade change or startup, material produced before the target composition is reached can require separation or reprocessing. The endpoint alone does not describe that transient.

At the same temperature, different current tank compositions can lead to different next responses. Learn a short state transition and apply it repeatedly to predict the full trajectory.



T = 330 K, τ = 5 min, C_Af = 1.2 mol/L; only the initial state differs.

State-transition model training and rollout evaluation

1. Current state, prescribed future inputs, fixed context, and sampling interval
2. State-transition MLP training on concentration increments
3. One-step versus 30 min rollout on held-out trajectories

Submission: the same model’s component errors in both evaluations, error versus horizon, and two temperature-plan trajectories. Optional extensions: RNN/LSTM and encoder-decoder.

Steady-state and state-transition maps

Week 2: [T, τ, C_Af] → the composition after equilibration.

Week 3: current composition + interval input + context → composition one interval later.

$$c_{k+1} = F_{\Delta t}\left(c_k, u_k; q\right)$$

Repeatedly applying this map produces a trajectory. A feedforward MLP can model a dynamic transition when its inputs contain the state needed for that transition.

An equilibrium predictor cannot account for how much A and B remain in the tank now. A transition model includes that current state, so the same temperature can produce different next states from different starting compositions.

State, future input, and fixed context

Quantity	Meaning	Unit / role
$c_k = [C_{A,k}, C_{B,k}, C_{C,k}]$	Current reactor composition	mol/L; state
$u_k = T_k$	Temperature over interval k	K; prescribed input
$q = [\tau, C_{Af}]$	Residence time and feed concentration	min, mol/L; fixed per trajectory
$\Delta t = 0.2$	Sampling interval	min; fixed for this model

Specify c_0 and the complete temperature sequence before rollout. The temperature plan is supplied by the exercise.

The reference CSTR

An ideal, perfectly mixed reactor has constant volume and equal inlet/outlet flow. Its consecutive first-order reactions are A → B → C with 1:1 stoichiometry. Feed contains A only. Temperature is prescribed; an energy balance is not included.

$$\frac{dC_A}{dt} = \frac{C_{Af} - C_A}{\tau} - k_1 C_A$$

$$\frac{dC_B}{dt} = k_1 C_A - \frac{C_B}{\tau} - k_2 C_B$$

$$\frac{dC_C}{dt} = k_2 C_B - \frac{C_C}{\tau}$$

$k_1(T) = 0.2 \exp[6000(1/330 - 1/T)]$, $k_2(T) = 0.1 \exp[5000(1/330 - 1/T)]$ in min^{-1}. At 330 K, $k_1 = 0.2$ and $k_2 = 0.1$ min^{-1}.

A continuous process, sampled at fixed times

For each interval [t_k, $t_k + \Delta t$), hold T_k constant. Integrate the balances over that interval to obtain c_{k+1}. The state stays continuous when the input switches.

30 min / 0.2 min = 150 transitions; 151 sampled states including the initial state. Temperature step at 10 min: interval index 50.

The learned map predicts a 0.2 min transition. Increasing the number of applications extends the horizon; it does not change the sampling interval.

The state just before the temperature change at 10 min initializes the next interval. Even with an instantaneous prescribed temperature change, concentrations are not replaced by their new equilibrium values. Compute their evolution over the next 0.2 min using the new rates.

Interval response of the reference model

With constant interval inputs, the balances are affine in the state. Write dc/dt = Mc + b. The steady state $c*$ satisfies $Mc* + b = 0$.

$$F_{\Delta t}\left(c, u; q\right) = c^* + e^{M\Delta t}\left(c - c^*\right)$$

The lab generates labels at the selected Δt from this analytic solution.

At c_0 = [1.2, 0, 0], T = 330 K, τ = 5 min, feed = 1.2 mol/L: $c_1 \approx$ [1.153870, 0.045672, 0.000458] mol/L.

Subtracting equilibrium removes the constant feed term, leaving deviations. The matrix exponential describes both their decay and transfer from A into B and C. At a changed input, recompute equilibrium and the matrix while retaining the current state.

Connecting the balance matrix to exponential response

At 330 K and τ=5, M has rows [−0.4,0,0], [0.2,−0.3,0], [0,0.1,−0.2], and b=[0.24,0,0]. Its second row gives the original B balance, $0.2C_A - 0.3C_B$.

Subtracting equilibrium c^* gives dz/dt=Mz. The scalar solution of dz/dt=−0.4z is exp(−0.4t)z(0); exp(Mt)z(0) extends this to coupled components.

$$e^{Mt} = I + Mt + \frac{(Mt)^2}{2!} + \frac{(Mt)^3}{3!} + \cdots$$

M^2 is a matrix product, not an elementwise power. Likewise, exp(Mt) is not an array of elementwise exponentials. Mt is dimensionless. Keeping only I+Mt gives one Euler step; later terms account for changing concentrations feeding back into reaction rates within the interval.

For z(0)=[0.6,−0.4,−0.2], Mz=[−0.24,0.24,0]. Thus c^*+(I+0.2M)z=[1.152,0.048,0], recovering the Euler calculation.

Concentration changes over the first 0.2 min

At [1.2,0,0], A inflow minus outflow is zero and its reaction consumption is 0.24 mol/(L·min). B forms at that rate; C initially has zero production rate because no B is yet present.

Quantity	Calculation	mol/L
Euler A	$1.2-0.2\times0.24$	1.152000
Exact A	$0.6+0.6\exp(-0.08)$	1.153870
B equilibrium-deviation term	$0.4-0.4\exp(-0.06)$	0.023294
Transfer from A into B	$0.12[\exp(-0.08)-\exp(-0.06)]/(-0.1)$	0.022378
Exact B	0.023294+0.022378	0.045672
Exact C	1.2−1.153870−0.045672	0.000458

Euler uses the initial rate throughout the interval. During the actual interval, A decreases, B forms, and C production begins. These changing rates explain its difference from the exact interval solution.

Integrating the effect of A on B

Write deviations $z_A = C_A - C_A^*$ and $z_B = C_B - C_B^*$, with $a = 1/\tau + k_1$ and $b = 1/\tau + k_2$. Then $z_A(t) = z_A(0)\exp(-at)$.

Multiply $z_B' + bz_B = k_1 z_A(0)\exp(-at)$ by $\exp(bt)$. The left side becomes the derivative of a product. Integrating from 0 to t gives:

$$e^{bt} z_B(t) - z_B(0) = k_1 z_A(0) \frac{e^{(b-a)t} - 1}{b - a}$$

Multiplying by $\exp(-bt)$ gives the transfer term $k_1 z_A(0)[\exp(-at) - \exp(-bt)]/(b-a)$. In the numerical calculation, 0.12 is $k_1 z_A(0) = 0.2 \times 0.6$, and −0.1 is $b - a = 0.3 - 0.4$.

If a=b, the integrand is 1 and its integral is t. Use $k_1 z_A(0)t\exp(-at)$ rather than evaluating 0/0.

Trajectory generation and transition-pair construction

Input ranges: 300-360 K, τ = 1-10 min, feed = 0.8-1.5 mol/L. Random temperature plans: six 5 min blocks. Initial composition for each trajectory: nonnegative components summing to feed.

Split	Trajectories	Transition rows
Train	60	9000
Validation	15	2250
Test	20	3000

Adjacent transition rows share states from the same trajectory. Random row splitting can put nearly the same trajectory in training and test. Splitting whole trajectories first preserves evaluation on unseen temperature plans and initial states.

States, inputs, and targets in the training data

Fields	Construction
Input v_k	[C_A,k, C_B,k, C_C,k, T_k, τ, C_Af]
Next-state label	c_k+1 from the reference interval response
Increment target	Δc_k = c_k+1 − c_k
Identification	split, trajectory ID, t_k

$$\Delta c_k = c_{k+1} - c_k$$

$$\hat{c}_{k+1} = c_k + \mu_\Delta + s_\Delta \odot g_\theta\left(\tilde{v}_k\right)$$

Target: concentration increment in mol/L

The first row is [1.2,0,0,330,5,1.2], with target increment approximately [−0.046130,0.045672,0.000458] mol/L. Learning an increment focuses on the local change; adding it to the current state is still required to obtain the next state.

The transition MLP and its scaling

Architecture: $6 \to 32 \to 32 \to 3$; ReLU hidden layers and a linear output. Parameters: $(6\times32 + 32) + (32\times32 + 32) + (32\times3 + 3) = 1379$.

Compute input and increment means/standard deviations from train rows. Apply those same statistics to validation, test, and rollout states.

MLP output: standardized increments. Restore each increment to mol/L before adding it to the current state.

Training means and scales define the coordinates in which the model interprets inputs. Recomputing them from predicted rollout states would map the same physical state to different numbers and change the learned input convention.

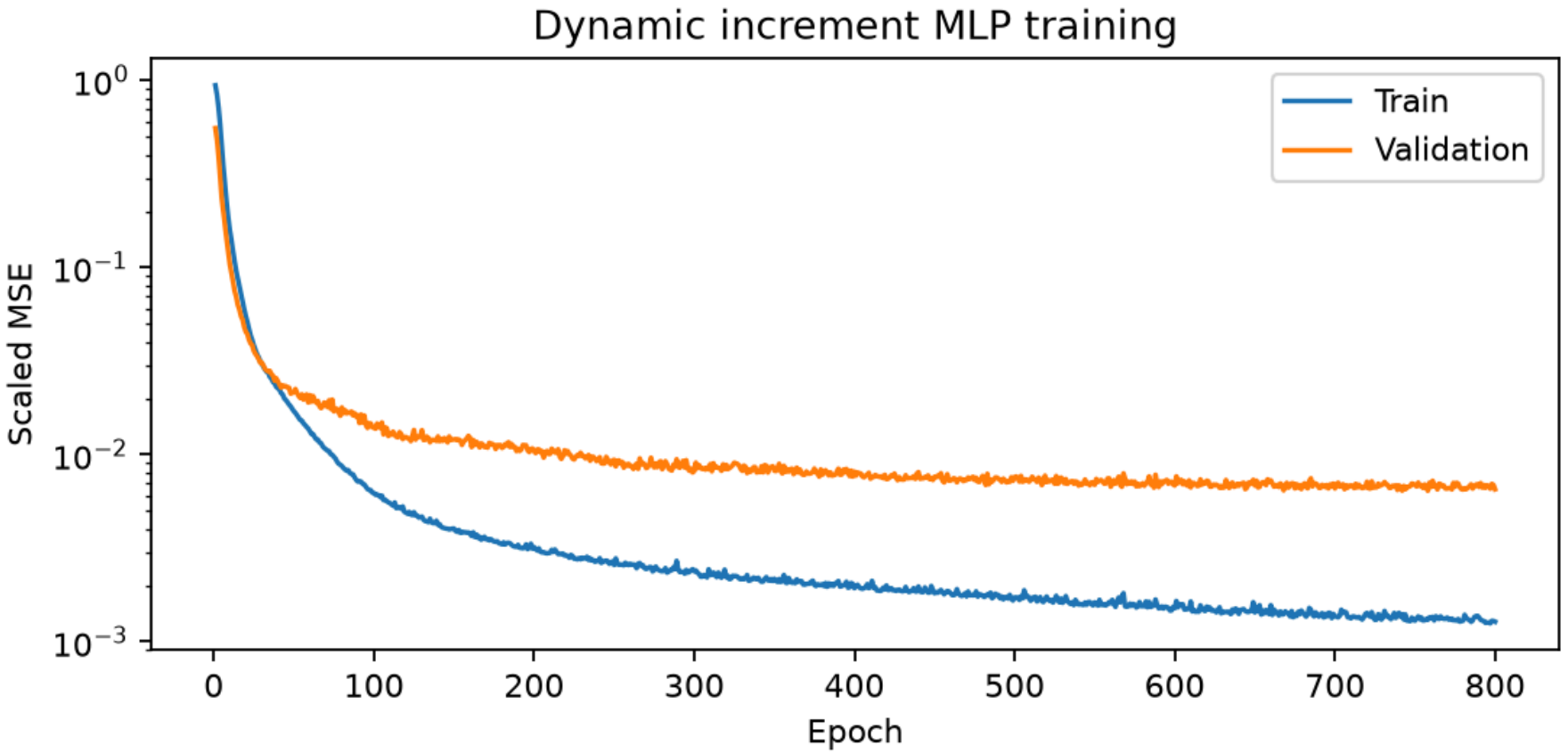
Train on reference current states

Each train input contains the reference current state. Minimize the mean squared standardized increment error.

$$L_1 = \frac{1}{3N} \sum_n \sum_j \left(g_\theta(\tilde{\mathrm{v}}_n)_j - \Delta\tilde{\mathrm{c}}_{n,j} \right)^2$$

Training: Adam, learning rate 0.001, batch size 512, up to 800 epochs. Weight selection: minimum validation loss. Test-trajectory evaluation after selection.

Training history



The horizontal axis counts epochs; the vertical axis is loss on standardized increments. Select weights at the minimum validation loss. A low value does not establish accurate recursive predictions, because this loss uses reference current states; the next evaluation tests rollout separately.

One-step and rollout evaluation

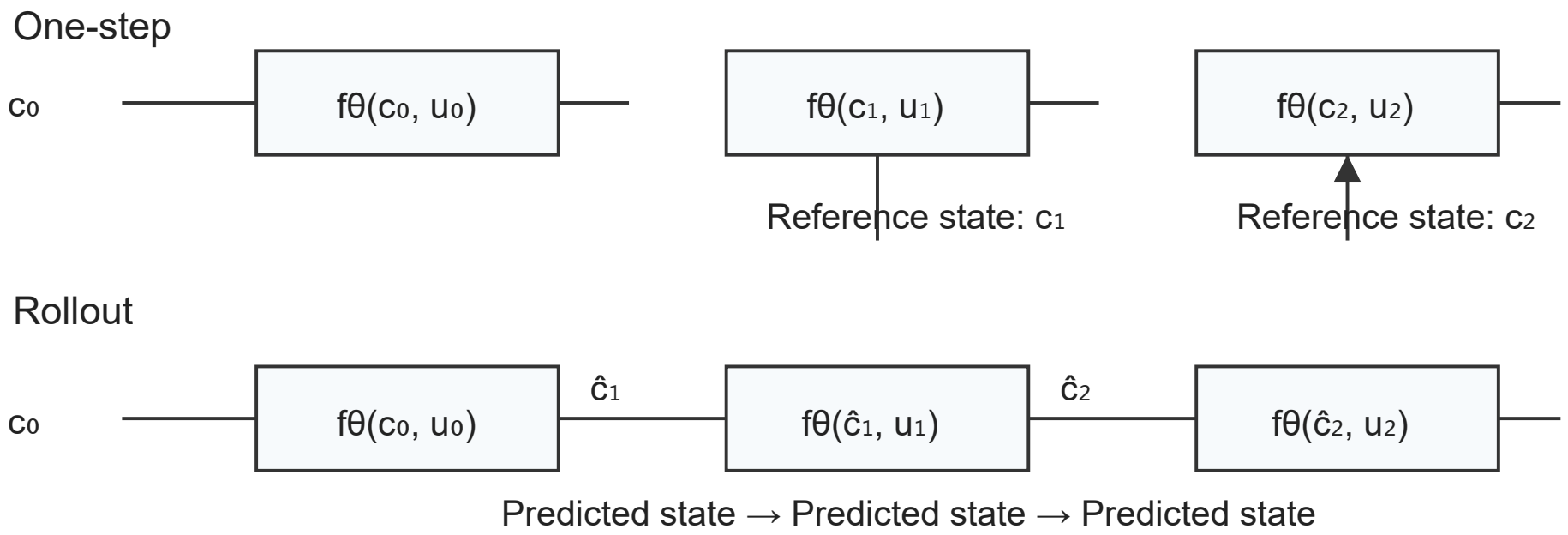
$$\text{one-step:}\hat{c}_{k+1} = f_\theta\left(c_k, u_k; q\right)$$
$$\text{rollout:}\hat{c}_{k+1} = f_\theta\left(\hat{c}_k, u_k; q\right), \hat{c}_0 = c_0$$

One-step: reset the input to the reference state at every time.

Rollout: start at c_0, then feed each predicted state into the next transition. Keep all future inputs and the context identical in both evaluations.

One-step evaluation receives a new reference state at every time, so it uses more state information than a forecast initialized only once.

Reference-state and predicted-state inputs



Both paths start from the same initial state. At the second transition, one-step uses reference c_1 while rollout uses its first prediction $\hat{c}_1$. The feedback arrow marks where the first error enters the next input.

Iterative computation of multistep rollout

```
pred = np.empty((len(controls) + 1, 3))
pred[0] = initial_state
for k, control in enumerate(controls):
    pred[k + 1] = dynamic_step(
        model, scaling, pred[k:k+1], control[None, :]
    )[0]
```

No reference state enters this loop after initialization. Evaluate all 150 predicted states.

pred[k:k+1] retains shape (1,3), and control[None,:] creates one control row. Index [0] extracts the returned state row. Reading pred[k] and writing pred[k+1] carries the previous prediction into the next iteration.

Propagation of one-step prediction error

Let e_k = ĉ_k − c_k. Add and subtract the prediction at the reference state:

$$e_{k+1} = f_\theta\left(\hat{c}_k, u_k\right) - f_\theta\left(c_k, u_k\right) + \delta_k$$
$$\delta_k = f_\theta\left(c_k, u_k\right) - F_{\Delta t}\left(c_k, u_k\right)$$

The first difference transports the existing state error through the learned map. δ_k is the local prediction error at the reference state. Even a small δ_k enters each successive prediction.

First compare the model at predicted and reference states to isolate the effect of input-state error. The remaining difference between model and process at the reference state is the new local error δ. Adding the two differences recovers the next-state error.

Accumulation of one-step bias: a worked example

For a scalar illustration, let the reference be F(c) = 0.9c and the learned map be f(c) = 0.9c + 0.001. Start with e_0 = 0.

$$e_{h+1} = 0.9e_h + 0.001$$

$$e_h = 0.001\frac{1 - 0.9^h}{1 - 0.9}$$

One-step bias is 0.001; at 10 recursive steps the error is about 0.00651. A contracting map limits the accumulation. A different state sensitivity can produce a different horizon pattern.

Intermediate states in the two evaluation protocols

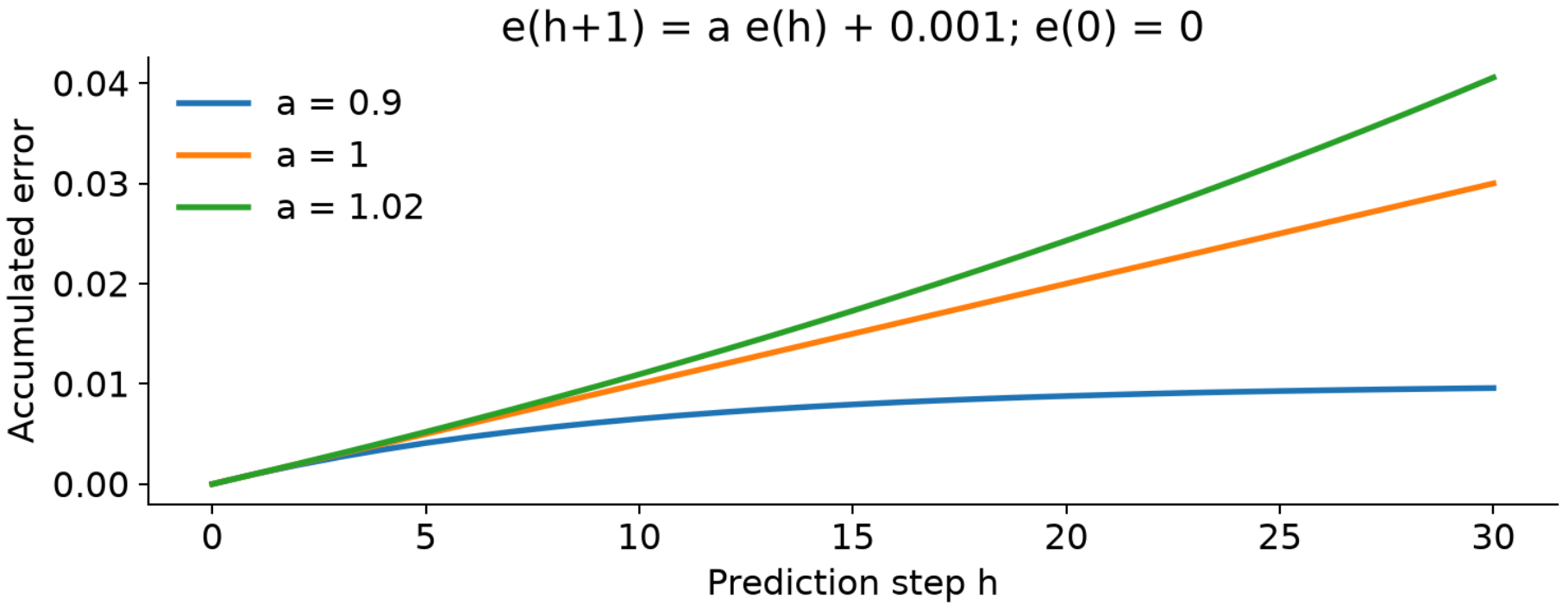
Use reference $F(c)=0.9c$, model $f(c)=0.9c+0.001$, and $c_0=1$. Values below are calculated from these scalar functions.

Step	Reference c	One-step prediction	Rollout prediction	Rollout error
0	1	Initial state	1	0
1	0.9	0.901	0.901	0.001
2	0.81	0.811	0.8119	0.0019
3	0.729	0.730	0.73171	0.00271

```
state = 1.0
for step in range(3):
    state = 0.9 * state + 0.001
    print(step + 1, state)
```

The second rollout computes $0.9\times0.901+0.001$. One-step receives reference 0.9 again and computes $0.9\times0.9+0.001$, so it does not inherit the previous error.

State sensitivity and error propagation



Actual rollout errors can increase, level off, or change sign.

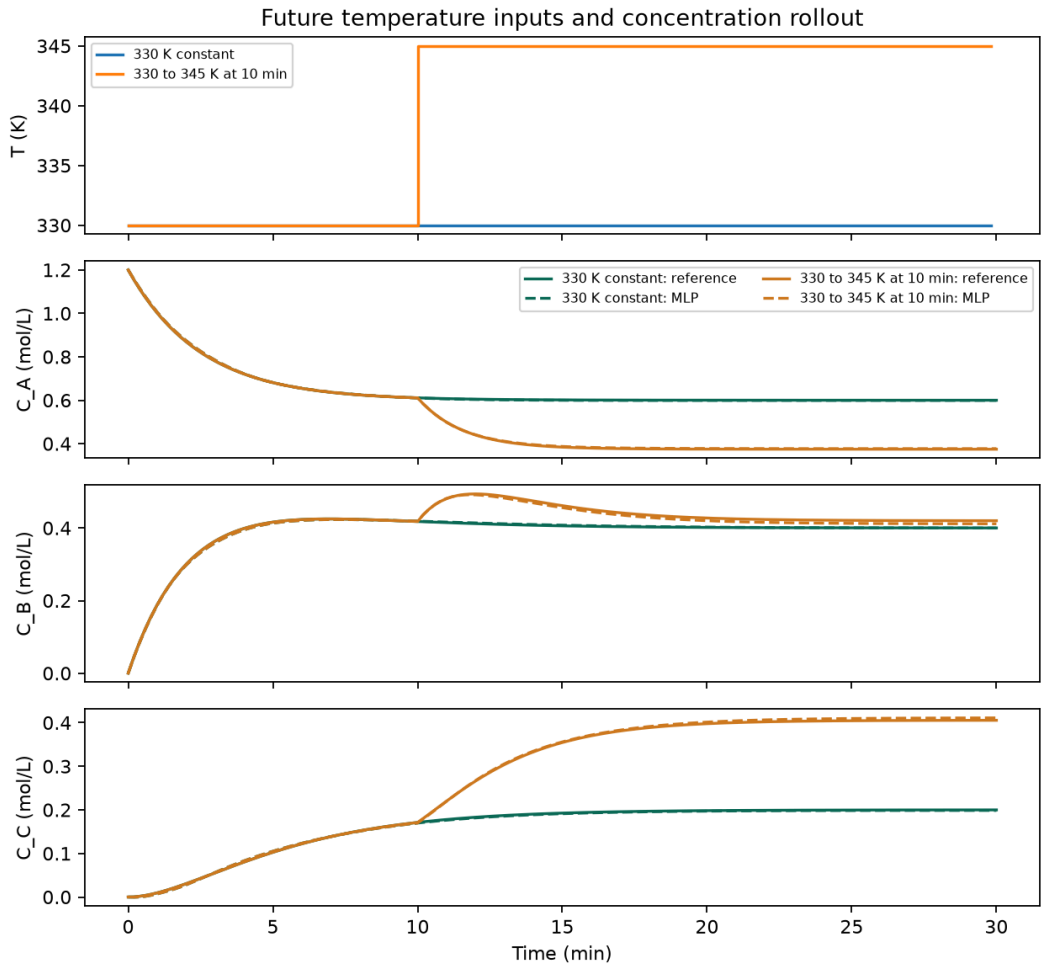
A sensitivity of 0.9 carries 90% of existing error into the next step. At 1, previous error remains and new bias keeps adding; above 1 it can be amplified. In a multivariable model, sensitivity depends on direction and state.

Two prescribed temperature plans

Setting	Constant plan	Step plan
Initial composition, mol/L	[1.2, 0, 0]	[1.2, 0, 0]
τ; C_Af	5 min; 1.2 mol/L	5 min; 1.2 mol/L
$0 \le t < 10$ min	330 K	330 K
$10 \le t \le 30$ min	330 K	345 K

Compare reference trajectories and the MLP rollout under exactly the same plans. Check when the curves separate and how they approach their new steady response.

A, B, and C over the full trajectory



Concentrations evolve continuously after the temperature change. Read A depletion, B formation, and conversion of B to C together. At the same time, the vertical difference between reference and predicted trajectories is that component's rollout error.

Compare error in physical units

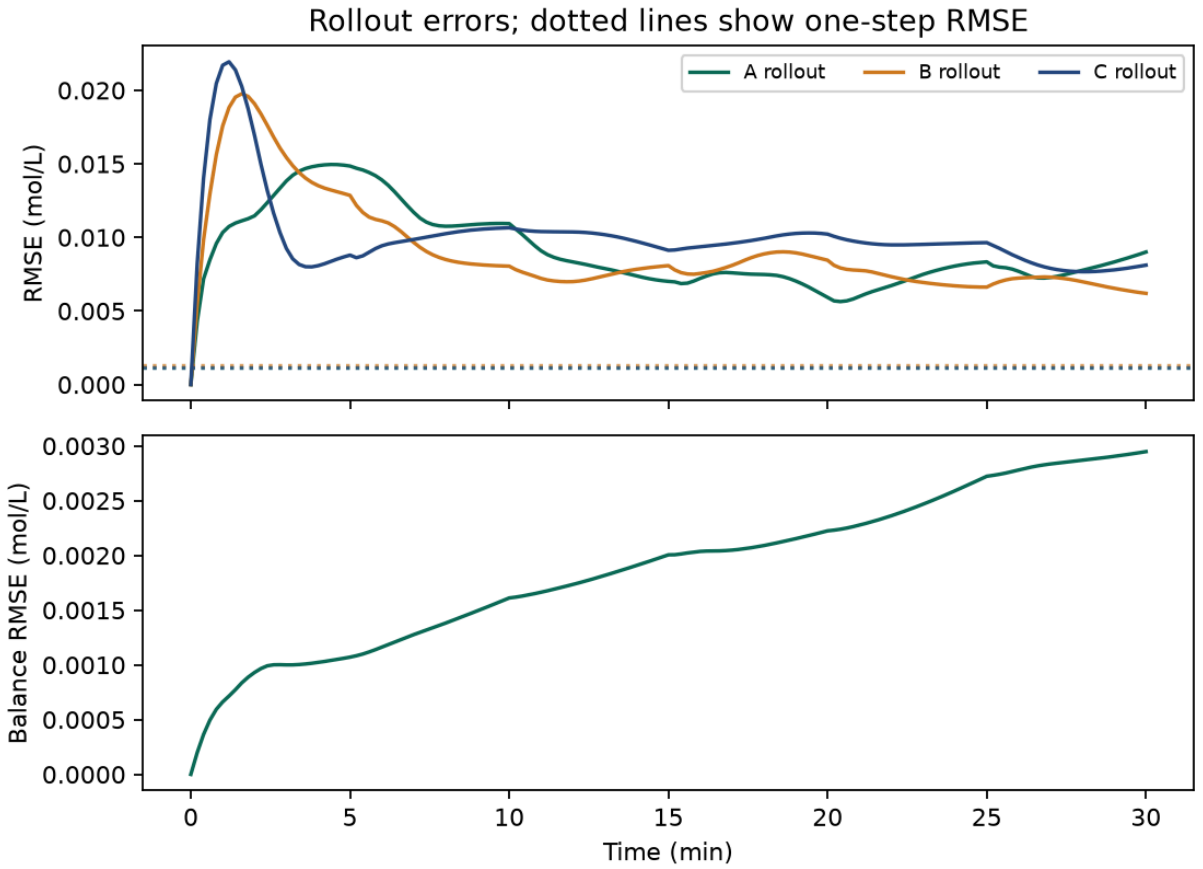
RMSE, mol/L	A	B	C
One-step	0.001170	0.001288	0.001163
Rollout	0.009465	0.009629	0.010425

Results from 20 held-out trajectories, 150 predicted times each. Rollout errors are larger in this stored run. Compute per-component RMSE across the 3000 predictions, then also inspect each horizon separately.

$$\mathrm{RMSE}_{j,h} = \sqrt{\frac{1}{N}\sum_{n}\left(\hat{C}_{n,h,j} - C_{n,h,j}\right)^2}$$

Here n indexes test trajectories, h the number of transitions, and j the component.

Read the error versus horizon



h=1 measures the first prediction from the initial state; h=150 measures the forecast after 30 min. Averaging the 20 test trajectories separately at each h reveals accumulation with horizon. A single RMSE over all times hides this shape.

Physical checks along the trajectory

$$S = C_A + C_B + C_C, \frac{dS}{dt} = \frac{C_{\mathrm{Af}} - S}{\tau}$$

$$S_{k+1} = C_{\mathrm{Af}} + \left(S_k - C_{\mathrm{Af}}\right) e^{-\Delta t / \tau}$$

With $S_0 = C_{Af}$ and fixed feed, the reference sum stays equal to feed. The learned outputs need not satisfy this relation.

Check, mol/L	One-step	Rollout
Sum-residual RMSE	0.000051	0.002014
Minimum prediction	-0.001152	-0.005166

Restoring increments and checking total concentration

Take state [0.6,0.4,0.2] mol/L, increment mean [0,0,0], scales [0.02,0.01,0.01] mol/L, and model output [−0.5,0.6,0.4].

Step	Value
Standardized output	[−0.5,0.6,0.4]
Multiply by scales	[−0.010,0.006,0.004] mol/L
Add to current state	[0.590,0.406,0.204] mol/L
Total concentration	0.590+0.406+0.204=1.200 mol/L

```
increment = d_mean + d_scale * standardized_increment
next_state = state + increment
```

These example increments sum to zero. A general MLP does not enforce that property, so check the sum and negative entries at each predicted state. When the initial total differs from feed, compare against the exact total-concentration transition rather than a fixed 1.2.

Final B concentrations under two temperature plans

B at 30 min, mol/L	Reference	MLP rollout
Constant 330 K	0.400091	0.399819
330 → 345 K at 10 min	0.420021	0.411526

Both calculations give more B at 30 min under the step plan. The MLP underestimates that endpoint by about 0.00850 mol/L.

A time-dependent product specification would require checking the relevant interval, not just this final sample.

Run the local lab and submit the comparison

The supplied lab bundle includes the reference, stored model, scaling, plots, and notebook.

```
python -m pip install -r requirements.txt
python week03_lab.py --output-dir week03-results
# Optional: train again with the same data construction
python week03_lab.py --train --output-dir week03-retrained
```

Submission: state/input definitions, split/scaling details, one-step and rollout RMSE, horizon plots, and two temperature-plan trajectories.

Exercises and reading

1. Derive dS/dt and state the condition for S to remain equal to feed.
2. Compute the first 0.2 min transition and compare it with forward Euler.
3. Explain why the same MLP receives different state inputs in the two evaluations.
4. Change an initial state while keeping the input plan fixed. Compare both reference and predicted trajectories.

reading companion

Optional: hidden states in RNNs and LSTMs

The core example supplies all three concentration states and prescribed temperature. If measurements omit relevant states, a history of observations and inputs can help reconstruct them.

$$h_k = \varphi\left(W_x x_k + W_h h_{k-1} + b\right)$$
$$\hat{y}_k = W_y h_k + b_y$$

An RNN updates a learned hidden state. An LSTM adds gated memory updates. Compare these models with the same trajectory split and forecast information.

Implementation: PyTorch RNN and LSTM.

For a scalar RNN with $\varphi=\tanh$, $W_x=1, W_h=0.5, b=0$ and $h_0=0$, observation $x_1=1$ gives $h_1=\tanh(1)\approx 0.762$. Even when $x_2=0$, $h_2=\tanh(0.5\times 0.762)\approx 0.363$ retains information from the past. An LSTM uses learned gates to control retention, addition and exposure of memory.

Optional: an encoder and a trajectory decoder

Encoder: summarize past observations and inputs into a state representation.

Decoder: predict the future concentration sequence using that representation and the prescribed future input plan. Ensure each future input reaches the corresponding prediction step.

Read Sutskever, Vinyals & Le (2014), Sections 1-2, for the sequence-to-sequence idea.

Original paper.